

Lecture 3

ESSLLI Course — *Introduction to Homotopy Type Theory / Univalent Foundations*

Tom de Jong

Additional reading

The citations refer to the file `Resources.pdf`. Related material can be found in [Rij25, Sections 12.1, 12.2; 14.1, 14.3, 14.4; 15.1]. See [Esc19, Sections 3.2.2; 3.34.2–3.34.4; 3.34.6] for a somewhat comparable treatment in Agda.

1. Logical propositions as *some* types

We will encode logical statements as types and a proof/witness of such a logical statement will be a term/element of that type. But which types should count as interpreting logical statements? For a type like $\mathbb{N}$, it seems unreasonable that it would interpret a logical statement, although it seems completely reasonable that it may act as a domain for a quantifier. On the other hand, types like $\mathbb{1}$ and $\mathbb{0}$ are easily imagined to interpret the logical statements $\top$ (true) and $\perp$ (false).

What distinguishes these types is that $\mathbb{1}$ and $\mathbb{0}$ both have **at most one element**. We call such types **propositions** (or **subsingletons**) and formally define for a type A , a new type

$$\text{is-prop}(A) := \prod (a, b : A), a = b$$

expressing that any two elements of A are identified, i.e. A has at most one element if we have an element of $\text{is-prop}(A)$.

From now on we will say and write “if $\text{is-prop}(A)$ ” rather than “if we have an element of $\text{is-prop}(A)$ ”, and similarly for other types.

Thinking classically for a moment, the only propositions are $\mathbb{0}$ and $\mathbb{1}$. Let’s check that these are actually propositions, i.e. that we have an element of $\text{is-prop}(\mathbb{0})$ and of $\text{is-prop}(\mathbb{1})$. For the former, we assume to be given $a, b : \mathbb{0}$ so that we are done by the elimination rule (anything goes if we have an element of $\mathbb{0}$). For the latter, the elimination rule says that it suffices to consider the case $\star = \star$ which has `refl` as an element.

We sometimes write

$$\text{Prop} := \sum (P : \mathcal{U}), \text{is-prop}(P)$$

sometimes with a subscript $\mathcal{U}$ to be more precise. Formally, an element of Prop is a pair of a type P and a proof that $\text{is-prop}(P)$, but we often omit the second component.

2. Some logical connectives

Conjunction

It makes sense to encode logical conjunction using the product type, but does this restrict to propositions? That is, given two propositions P and Q , is $P \times Q$ again a proposition? The answer is **yes**. To see this, suppose we are given two elements $(p, q), (p', q') : P \times Q$. We must show that they are equal.

This raises an immediate question: **When are two elements of a product type equal?**

Answer: Such elements are equal exactly when they are componentwise equal, i.e. for general types A and B , we have $(a, b) = (a', b')$ if and only if $a = a'$ and $b = b'$. More precisely, we can define functions back-and-forth

$$(a, b) = (a', b') \longleftrightarrow (a = a') \times (b = b')$$

given by path induction:

$$\begin{aligned} \text{refl}((a, b)) &\mapsto (\text{refl}(a), \text{refl}(b)); \\ \text{refl}((a, b)) &\leftarrow (\text{refl}(a), \text{refl}(b)). \end{aligned}$$

In fact, these maps form an equivalence, but let's revisit that later.

If P and Q are propositions, then $p = p'$ and $q = q'$, so $(p, q) = (p', q')$, as desired, and $P \times Q$ is again a proposition.

Implication

For implication, we would like to use function types. Suppose P and Q are propositions. Is $P \rightarrow Q$ a proposition?

This raises an immediate question: **When are two elements of a function type equal?**

Answer: We will *postulate* that two functions are equal when they are pointwise equal, i.e. by adding the **function extensionality axiom**:

$$\left(\prod (a : A), f(a) = g(a) \right) \longrightarrow f = g$$

for all functions $f, g : A \rightarrow B$.

This is **not** provable in plain MLTT.

In fact, this map then becomes an equivalence, but let's revisit that later.

Now, if P and Q are propositions, then so is $P \rightarrow Q$, for if we have $f, g : P \rightarrow Q$, then by function extensionality, we only need to show $f(p) = g(p)$ for all $p : P$, but Q is a proposition, so we are done.

Notice that we only needed the codomain in the function type to be a proposition. This observations helps in the following.

Universal quantification

If we upgrade the function extensionality axiom to also govern *dependent* functions, then $\prod (x : A), P(x)$ is a proposition whenever $P(x)$ is a proposition for every $x : A$ (even when A is not a proposition). We then encode $\forall$ as $\prod$.

E.g. the type $\prod (n, m : \mathbb{N}), n + m = m + n$ expressing the commutativity of addition on natural numbers is a proposition. Actually, this needs that $x \mapsto y$ is a proposition which we will show next time ($\mathbb{N}$ is a so-called *set* or 0-type).

Negation

As usual in intuitionistic/constructive logic, we *define* negation as implying false, so that we put

$$\neg A := A \rightarrow \mathbb{0}.$$

Since $\mathbb{0}$ is a proposition, this type is a proposition (under function extensionality).

3. More logical connectives?

Disjunction?

It may seem tempting to encode disjunction $\vee$ using coproduct types, but $P + Q$ need not be a proposition. The simplest example is $\mathbb{1} + \mathbb{1}$ which is the definition of 2 with distinct elements 0 and 1 .

This is a nice opportunity to show that $0 \neq 1$, i.e. that $0 \stackrel{2}{=} 1 \rightarrow 0$. Suppose we have $p : 0 = 1$. We need to construct an element of 0 . Consider the function $C : 2 \rightarrow \mathcal{U}_0$ given by $C(0) := 0$ and $C(1) := 1$. Then we have $\star : C(1)$, so that $\text{transport}^C(\star, p^{-1}) : C(0) \equiv 0$. ■

Existential quantification?

While we can show that if P and $Q(p)$ are propositions, then so is $\sum(p : P), Q(p)$, this does not account for encoding statements of the form $\exists(n \in \mathbb{N}), Q(n)$ as e.g. $\sum(n : \mathbb{N}), n + 0 = n$ has $\mathbb{N}$ -many elements.

4. Propositional truncation

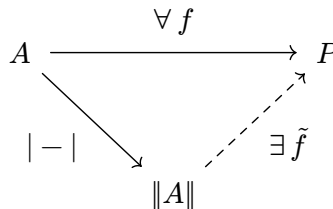
Our solution will be to postulate a new operation called the **propositional truncation**. It can be given as a higher inductive type (HIT), an idea that we will revisit in the last lecture.

For now, we just assume to be given

1. an operation $\| - \|$ on types such that $\|A\|$ is a proposition for all types A ,
2. a function $| - | : A \rightarrow \|A\|$ for every type A such that
for every *proposition* P and every function $f : A \rightarrow P$, we have $\tilde{f} : \|A\| \rightarrow P$ such that $\tilde{f}(|a|) = f(a)$ for all $a : A$.

This condition is known as the **universal property** of the propositional truncation.

We summarize the requirements with the following diagram which is required to commute, i.e. the ways of going round the diagram must agree.



By function extensionality and the assumption that P is a proposition, the map $\tilde{f}$ is **necessarily unique**: any two maps with codomain P must be equal.

Note that we have described the propositional truncation via a universal property / mapping out property. This idea from category theory is very powerful and we will employ this methodology for other types too (even for a simple type like $\mathbb{N}$) in the last lecture.

The idea is that $\|A\|$ is like A but we've identified all of its elements to make it a proposition. However, the universal property says that *to propositions*, the types $\|A\|$ and A behave the same.

We now define

$$P \vee Q := \|P + Q\| \quad \text{and} \quad \exists(x : A), P(x) := \left\| \sum(x : A), P(x) \right\|.$$

Example: Suppose we wanted to show that $(\exists(x : A), P(x)) \rightarrow Q$ where Q is a proposition. By the universal property, it suffices to construct a function $(\sum(x : A), P(x)) \rightarrow Q$, so that we may assume to have an element (a, p) with $a : A$ and $p : P(a)$, as familiar from logic.

The truncation does NOT erase

We give two examples to show that the (common?) intuition that the propositional truncation erases information is not accurate.

- Let P be a proposition. Then one can show (using a technique similar to the proof that $0 \neq 1$) that the type $P + \neg P$ is a proposition ([exercise!](#)). This immediately gives a function

$$(P \vee \neg P) \longrightarrow (P + \neg P)$$

so, given $P \vee \neg P$, we can really tell whether P or $\neg P$ holds by inspecting whether the element of the type $P + \neg P$ starts with `inl` or `inr`.

- (Escardó) Suppose that P is a dependent type on $\mathbb{N}$ such that every $P(n)$ is a proposition and moreover **decidable**, by which we mean that we have $P(n) + \neg P(n)$. While the type $\sum(n : \mathbb{N}), P(n)$ is not a proposition since there may be many n 's that satisfy P , we can still define a function

$$(\exists(n : \mathbb{N}), P(n)) \longrightarrow (\sum(n : \mathbb{N}), P(n))$$

out of the truncation as follows.

We consider the type of *least* witnesses of P , i.e.

$$Q := \sum(n : \mathbb{N}), P(n) \times \left(\prod(k : \mathbb{N}), P(k) \rightarrow n \leq k \right).$$

This type can be shown to be a proposition. Moreover, since P is a decidable predicate, we can construct a “search” function $\sum(n : \mathbb{N}), P(n) \rightarrow Q$ that yields the least n with $P(n)$. Hence, we get a commutative diagram

$$\begin{array}{ccccc}
 & & \exists(n : \mathbb{N}), P(n) & & \\
 & \nearrow \text{“}|- \text{”} & & \searrow & \\
 \sum(n : \mathbb{N}), P(n) & \xrightarrow{\text{“search for least”}} & Q & \xrightarrow{\text{“forget least proof”}} & \sum(n : \mathbb{N}), P(n)
 \end{array}$$

and our desired map $(\exists(n : \mathbb{N}), P(n)) \rightarrow (\sum(n : \mathbb{N}), P(n))$ by composing.

NOTE: This last example shows that making a type *more informative* can make it a proposition. This makes clear that propositions in the HoTT sense are **very different** from the (syntactic) propositions in the Calculus of (Inductive) Constructions that you may know from Rocq (formerly, Coq) or Lean.

5. Data vs property

In general, types encode **data**, whereas propositions encode **properties**. For instance, for a type A , an element $a : A$ is a **point** of A and is data, while $t : \|A\|$ just says that A is **inhabited**. Subscribers to classical logic would say **nonempty**, which we reserve for (an element of) $\neg\neg A$ (which we recall unfolds to $(A \rightarrow \mathbb{0}) \rightarrow \mathbb{0}$).

If you adopt the **law of excluded middle**, which we define as $\prod(P : \text{Prop}), P + \neg P$, then you can show that $\neg\neg A \longleftrightarrow \|A\|$ ([exercise!](#)).

In general we only have $\|A\| \rightarrow \neg\neg A$, but not the converse.

Terminology

Many people, following the HoTT Book, pronounce $\|A\|$ as *merely* A , and $\exists(x : A), P(x)$ as “*there merely exists*” to differentiate from $\sum(x : A), P(x)$. Like Escardó, I’m not a big fan of this. For one thing it deviates from standard mathematical practice. Instead I will write/say “there exists” for $\exists$, and sometimes

“there exists an *unspecified*” for emphasis. For Σ , I will say/write “we have” or “there exists / we have a *specified*”, or especially for iterated Σ -types I will use the phrase “together with”.

6. Important intermezzo: equivalences

Definition. An **equivalence** is a function $f : A \rightarrow B$ that has a left and a right inverse, i.e.

$$\text{is-equiv}(f) := \left(\sum (g : B \rightarrow A), \prod (b : B), f(g(b)) = b \right) \times \left(\sum (h : B \rightarrow A), \prod (a : A), h(f(a)) = a \right).$$

It is natural to wonder why we don’t just ask for a single map that is 2-sided inverse to f (like an isomorphism). The reason is that while $\text{is-equiv}(f)$ is a proposition, the other type need not be. We will return to this in the next lecture.

If f is an equivalence, then we will write f^{-1} for the first component (called g) above). This is justified by the fact that, besides $f(g(b)) = b$ for all $b : B$, we also have $g(f(a)) = a$. Indeed, given $((g, p), (h, q)) : \text{is-equiv}(f)$, we see that

$$g(f(a)) \xrightarrow{q_{g(f(a))}^{-1}} h(f(g(f(a)))) \xrightarrow{\text{ap}_h(p_{f(a)})} h(f(a)) \xrightarrow{q_a} a.$$

(We could also have chosen h to serve as f^{-1} .)

Remark. Note that if P and Q are propositions, then $f : P \rightarrow Q$ is an equivalence as soon as we have $Q \rightarrow P$, because the requirements that the maps compose to the identities are trivially satisfied as P and Q are both propositions.

We will say that **logically equivalent** propositions are equivalent.

7. An example of the need for $\exists$: defining the image of a function

In set theory, the **image of a function** $f : A \rightarrow B$ is defined as the subset $\{b \in B \mid \exists a \in A, f(a) = b\}$ of B .

A naive translation—where existence is encoded via Σ -types—of the image into type theory as the type

$$\sum (b : B), \sum (a : A), f(a) = b$$

is inadequate. This is because we have an equivalence $(\sum (b : B), \sum (a : A), f(a) = b) \simeq A$ given by

$$(b, a, p) \mapsto a \quad \text{and} \quad (f(a), a, \text{refl}) \leftarrow a.$$

One round-trip clearly composes to id_A , while we also have $(b, a, p) = (f(a), a, \text{refl})$ by path induction on p .

So the “image” would just be the domain, but this is clearly wrong, e.g. for $f : \mathbb{2} \rightarrow \mathbb{1}$.

However, the type

$$\sum (b : B), \exists (a : A), f(a) = b$$

works as it should and is the correct definition of the **image** of f .

A **subtype** of a type X is $\sum (x : X), P(x)$ where $P(x)$ is a proposition for every $x : X$. Note that the image of $f : A \rightarrow B$ is a subtype of B .